Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 220 (2023) 608–615

Procedia Computer Science

www.elsevier.com/locate/procedia

The 6th International Conference on Emerging Data and Industry (EDI40), March 15-17, 2023, Leuven, Belgium

Maintenance Cost of Software Ecosystem Updates

Solomon Berhe[a,*], Marc Maynard[b], Foutse Khomh[c]

[a] *releasetrain.io, Stuttgart 70376, Germany*
[b] *Data Independence LLC, Ellington CT 06029, USA*
[c] *Polytechnique Montréal, Montréal, QC, H3C 3A7, Canada*

Abstract

Over the past decades, software domains have witnessed a trend towards faster software release cycles, an increase of software components, and their connectivity. Examples include products in Industry 4.0 domains such as health care, retail, or mobility, in which, technically, software components are connected to a software ecosystem. Faster release cycles and high connectivity make many software ecosystem updates an expensive, arduous, and risky maintenance task. This paper leverages software release metadata to quantify the maintenance cost of software ecosystem updates. In particular, the data is applied to model a software ecosystem graph and subsequent calculate its maintenance cost. To illustrate, the model is prototyped and evaluated in three Industry 4.0 use cases.

© 2023 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the Conference Program Chairs

Keywords: software; ecosystem; maintenance;

1. Introduction

Over the past decades, software domains have witnessed a trend towards faster software release cycles, an increase of software components, and their connectivity. As a result, many organizations adapted a transition towards an agile software engineering process [1]. This trend is not merely limited to software domains, but also includes traditionally hardware domains leveraging the advantages of Industry 4.0 [2]. Main factors include the broad availability of portable and connectivity enabled technology, such as Raspberry Pi [3] and mobile phones [4]. This technology led many organizations to apply or develop Industry 4.0 services [5], to respond faster to market feedback [6], and to transition towards an agile software engineering process [1]. Technically, software domains such as Industry 4.0 span software ecosystems that include a variety of interdependent software components, including: *firmware, operating systems, programming languages, libraries, frameworks, databases, browsers*. These software components are often

* Corresponding author. Tel.: 0049-15752440011;
E-mail address: solomon.berhe@gmail.com

1877-0509 © 2023 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the Conference Program Chairs
10.1016/j.procs.2023.03.077

developed, operated, and updated independently by separate organizations and component teams. As a result, Industry 4.0 application software ecosystem teams are left with the potentially expensive, arduous, and risky maintenance task to timely evaluate the impact of software updates [7, 8, 9]. Three common options are: a) ignore an update, b) automate an update, or c) evaluate the impact of an update. An update ignored invites to potentially costly exploits [10, 11]. For example, in 2021, Apache has classified the severity of multiple Log4Shell common vulnerabilities exposures (CVE) with high on the Common Vulnerability Scoring System [12]. Since this software component is part of the standard commons Java library, not only most Java based applications were affected but also dependent software components within its ecosystem. In this example, an update ignored can lead to thousands of scan and exploitation attempts on the same day, leading to economic risks [12, 13, 14]. Likewise, updates that deprecate a current component version can not be ignored for too long, since they may not be supported anymore [15]. An automated update ensures that software components have the latest version. Due to dependencies between software components, automatically updating to a major breaking or defect version may lead to breaking the software ecosystem [16, 17]. Neither ignoring, delaying, nor automating software updates guarantees maintaining a secure and working software ecosystem [18, 19]. In any phase of the software engineering life cycle, for potentially each software update available, ecosystem teams must address questions, such as:

1. Q1: What ecosystem components are affected by an update?
2. Q2: What ecosystem teams are required to address an update?
3. Q3: What is the maintenance cost of an ecosystem?

To clearly contrast, these ecosystem questions exclude the maintenance cost of technically performing a software update. For example, updating an operating system may technically be more complex than updating a library, however, generally, both updates can have the same ecosystem effort. The questions address efforts related to impact evaluating and coordinating each update. As a result, for example, the update time of an operating system can be relevant with regard to performing related ecosystem maintenance tasks, such as coordination with customer service or other stakeholders. Ecosystem teams are often left with the difficult efforts of addressing these types of questions [8]. To support this task, this paper proposes to leverage public software release metadata. In particular, the release version, release channel, release date, and release notes commonly include useful information to automatically evaluate the ecosystem impact of a software update [18]. Towards this objective, Section 2 will provide an overview of related work on software ecosystem maintenance cost. Section 3 will present a software release data-driven model towards generating a graph representation of a software ecosystem and subsequent calculate its maintenance cost. In Section 4, the model prototype implementation is illustrated and applied to three Industry 4.0 use cases, followed by a discussion in Section 5. Lastly, Section 6 will provide concluding remarks and future work.

2. Background and Related Work

In 1983, the American National Standards Institute (ANSI) and the Institute of Electrical and Electronics Engineers (IEEE) 729 standard defined software maintenance as: *modification of a software product after delivery to correct faults, to improve performance or other attributes, or to adapt the product to a changed environment*[20]. Based on this definition, in 1993 researchers performed a seminal literature review on software maintenance cost. At the time, with less Free and Open Source Software, application reviewed were mostly written by more interdependent teams [21]. Therefore, maintenance cost variables considered focused mostly on the programming and team level and none on third party related software component updates [20]. Later, in 2014, researchers did a similar review to address the relationship between software complexity and its maintenance cost. The research clearly acknowledges the need to consider software component update efforts as part of the maintenance cost and illustrates it with growing maintenance costs of operating systems leveraging variables such as lines of code, team sizes, and budget costs [22]. In contrast, the work presented herein applies software update related variables to calculate the maintenance cost on a software ecosystem level. Towards the same objective, in 2021, the IEEE standard extended the maintenance concepts and definitions for system maintenance, which includes *relevant external dependencies* [23, 24, 25].

With regard to software ecosystems, the following related contributions have been performed. In 2003, the seminal book on software ecosystems covers the organizational, market, and social evolution of software products [26].

It illustrates the growing trend towards multi-independent stakeholders producing and consuming software products. In 2009, researchers proposed a tool to technically reverse engineer the evolution of software ecosystems [27]. They leverage open source software release dates as time points to reverse engineer developer collaboration, number of commits by project, and software size by project in an ecosystem. In contrast to the work herein, their research is focused on modeling ecosystems with public code repositories and static ecosystem software projects. In 2010, researchers acknowledge the need for more ecosystem research and address the question: *"What key defining characteristics can be used to describe a software ecosystem in order to support the central hub of the software ecosystem in its decision making process?"* [28]. Their results present a generic Software Ecosystem Strategy Assessment Model, in which release planning is the closest characteristic to this paper [28]. In contrast, this paper leverages release notes data to model a software ecosystem, its maintenance cost, and applies it to prototype use cases. In 2013, researchers model software ecosystem with focus business value chain leveraging business roles data [29].

With regard to standardizing the tracking and collaboration of consolidated software update data, in 2022 open source developers have released the Open Source Vulnerability (OSV) format. With OSV it is possible to standardize the exchange format of CVE updates that impact open source software [30]. Similarly, in 2018, researchers leverage software package manager to track dependencies between software components [31]. Relatedly, in [30], the focus is on one specific open source package manager. In contrast, the model and prototype use cases presented in this paper are able to leverage any software update including commercial, closed source, and that is also independent of CVEs.

With regard to meeting the demands related towards faster release cycles, more component connectivity, and dependency, much research has been already done [1, 2, 16, 32]. Here it is worth highlighting research that not only addresses technical challenges, but also challenges related to people, organizational culture, processes, and scalability. These characteristics potentially adversely impact the maintenance cost on evaluating software updates on an ecosystem level [33]. Similarly, in 2017, researchers study the difficult decision-making process in agile software environments [34]. Related to these non-technical challenges, the model presented herein suggests the ecosystem specific definition of weights for ecosystem links, components, as well as component version types. Agile teams often leverage a generally experience based approach for addressing effort estimations [35, 36]. The maintenance cost model presented herein complements the generally experience and sprint based estimations leveraging release notes metadata.

In closing, the major contrast to previous research is the modeling and usage of broadly and publicly available software release metadata. Since many Industry 4.0 software ecosystems span a variety of commercial, open source, and closed source software components, we argue that software release metadata is a good candidate to realistically model and calculate their maintenance cost with respect to its ecosystem level software update efforts.

3. Model

This section details the definition of a graph-based software ecosystem model as well as the definition of the maintenance cost. The model is kept generic to meet the demands of various ecosystem software, people, organizational culture, processes, and scalability [33]. To keep the model applicable to many Industry 4.0 domains, nodes model any type of software component that supports release notes metadata. For example, a node can range from a small library to an entire programming language, database, or operating system. For each component node, a set of tags semantically describes a component. To illustrate, the component localstack is described with the tags: *aws, testing, continuous-integration, developer-tools, python, cloud*. Edges model relationships between software components and are kept generic as well. In the model, an edge is added to the ecosystem if two components share at least one description tag. Depending on the Industry 4.0 ecosystem, component semantic description tags can be customized, such that edges may for example model dependency, association, aggregation, or composition. The maintenance cost model is applied to the ecosystem and each component. More specifically, the model provides a maintenance cost for both, the ecosystem and ecosystem component. The cost of a component is the sum of a) the number of its edges and b) the number of its versions. To allow better customization, each component, edge, and version type can be weighted. In the model the version type is kept generic, however the prototype applies weights for the version types: *major, minor, patch, CVE*. The maintenance cost of the ecosystem is the sum of all its component costs.

3.1. Definition Software Ecosystem

1. $Eco^a = (N^a, E^a)$ is a software ecosystem graph.
2. $N^a = \bigcup c_i^a$ is the set of software component nodes in Eco^a.
3. $E^a = \bigcup (c_i^a, c_j^a)$, where $TAG(c_i^a) \cap TAG(c_j^a) \neq \emptyset$ is the set of edges in Eco^a.
4. $TAG(c_i^a) = \bigcup tag_{ij}^a$ is the set of semantic description tags in c_i^a.

3.2. Definition Maintenance Cost

4. $Cost(c_i^a) = w(component) + w(version) + w(links) * |c_i^a| + \sum_{versiontype} w(versiontype) * |versiontype(c_i^a)|$
5. $Cost(Eco^a) = \sum_i Cost(c_i^a)$

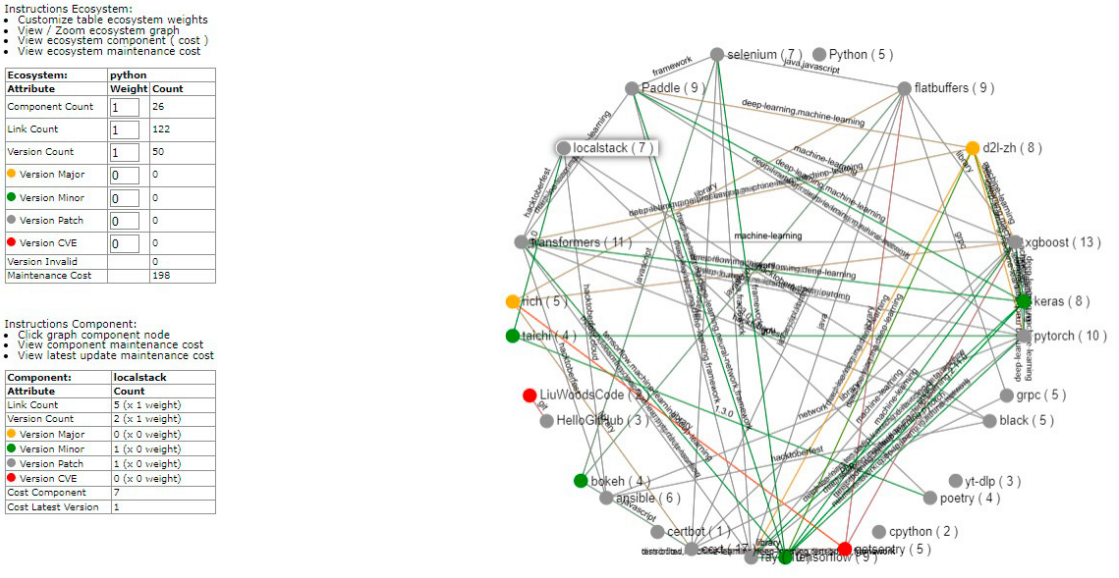
4. Prototype Use Case Evaluation

To evaluate the model, this section will present a prototype implementation and its application in three use cases.

4.1. Prototype

In the prototype implementation the release notes metadata to model the software component nodes are gathered from different online software release notes web documents [18]. As of January 2023, about 2,832 software components and 20,258 component versions are stored. The data is queried for semantic versioning information in both the version number and the version description and only added if its type is either a major, minor, patch or cve[37]. Figure 1 illustrates the Python ecosystem implementation. In this example, each component (26), link (122), and version (50) has weight 1, while version types such as major, minor, patch, and cve are not additionally weighted and have weight 0. The overall ecosystem maintenance cost is the sum of all positive weighted attributes (198) and the overall component maintenance cost is the sum of the positive weighted component related attribute (localstack = 7), while the maintenance cost of the latest version is the sum of the positive weighted version related attributes (localstack latest = 1).

Fig. 1. Model Prototype - Python Ecosystem / Localstack Component



4.2. Use Case Scenario

Based on the model for ecosystem graphs and maintenance cost, this section will present several Industry 4.0 use cases to illustrate different examples on how release notes metadata can be applied to answer the questions Q1 - Q3. For the first question, namely *What ecosystem components are affected by an update?*, a health care use case with stricter data protection and security requirements will be applied. For the second question, namely *What ecosystem teams are required to address an update?*, a small startup use case with limited testing capabilities will be presented. Lastly, for the third question, namely *What is the maintenance cost of an ecosystem?* an automotive use case with slower update cycles will be illustrated. Each use case will apply different weights to model different maintenance efforts. To normalize, for each use case the last 50 software updates will span the software ecosystem graph for review. The data to reproduce the cases can be retrieved from https://releasetrain.io/edi40-2023/.

4.2.1. Use Case 1 - Java Ecosystem in Health Care Domain

A health care software ecosystem is implemented with Java as the main programming language. It includes both, internal and external components. For the ecosystem team, careful impact evaluation and fast patching of CVEs are a main priority to ensure there is no risk with regard to patient data. On Dec/27/2022 two Java CVEs, namely CVE-2022-41966 and CVE-2022-41967, were released. Due to an upcoming holiday season most teams are on vacation next week. As a result, the ecosystem team shall request updating the higher risk CVE by addressing the question: *What ecosystem components are potentially more affected by each update?*. After generating the graph and cost model, table 1 illustrates CVE-2022-41967 has the potentially higher impact and maintenance cost since it is linked to more (3) software components. This could indicate potential dependencies and requires a more thorough review at an ecosystem level in coordination with the other related component teams. As a result, CVE-2022-41967 shall be addressed before next week.

Attribute	Weight	Value	Note
Ecosystem Components ($\|N^{Java}\|$)	1×29	29	
Ecosystem Edges ($\|E^{Java}\|$)	3×28	84	weight(links) = 3 to model higher impact/cost
Versions	1×50	50	
Version Major	0	0	
Version Minor	0	0	
Version Patch	0	0	
Version CVE	3×3	9	weight(cve) = 3 to model higher impact/cost
Cost(Eco^{Java})		172	Sum of components, edges, and CVE versions
Cost($c^{Java}_{CVE-2022-41966}$)		3	Potentially lower maintenance cost (1 edge)
Cost($c^{Java}_{CVE-2022-41967}$)		18	Potentially higher maintenance cost (3 edges)

Table 1. Use Case 1: Ecosystem Eco^{Java} Maintenance Cost of CVE Software Updates.

4.2.2. Use Case 2 - Testing Team by Ecosystem Maintenance Cost

An Industry 4.0 startup leverages the Python ecosystem to develop a software product. Due to limited testing capacity, they would like to optimize where to invest the main testing team effort. They assume patch and minor updates require more component testing, whereas major and CVE updates require more integration and system testing. Towards this objective, they need to address the question: *What ecosystem testing teams are required to address an update?*. Figure 1 and Table 2 illustrate based on the last 50 Python related software updates, component testing relevant updates (minor and patch) make up the 46/50 versions and integration testing relevant updates (major and CVE) make up 4/50 versions. As a result, based on this data, the main testing efforts are invested in component testing.

Attribute	Weight	Value	Note
Major Update	1	2	Integration Testing Required
CVE Updates	1	2	Integration Testing Required
Minor Updates	1	10	Component Testing Required
Patch Updates	1	36	Component Testing Required

Table 2. Use Case 2: Testing Team by Ecosystem Maintenance Cost.

4.2.3. Use Case 3 - Software Stack with Minimal Ecosystem Maintenance Cost

Many Industry 4.0 domains are based on traditionally hardware producing companies, such as automotive, bicycles, or scooters with much slower hardware release cycles. As a result, software updates are often impeded by hardware related constraints, leading to infrequent software updates. In this use case, an automotive domain ecosystem team is constraint to perform yearly software updates. To address this impediment, when planning the software stack their overall objective is to pick an ecosystem with the least maintenance cost and address the question: *What is the maintenance cost of a software stack ecosystem?*. The stacks that shall be evaluated include:

1. Linux, Apache, MySQL,PHP (LAMP)
2. Windows, Apache, MySQL,PHP (WAMP)
3. MongoDb, Express, Angular, NodeJs (MEAN)
4. MongoDb, Express, React, NodeJs (MERN)
5. MongoDb, Express, Vue, NodeJs (MEVN)

Based on the last 50 version updates, the MEAN stack has the lowest maintenance cost, because the number of components and edges is the lowest. From an ecosystem view, this leads to less component updates and less impact on other components at an ecosystem level.

Attribute	Weight	Value	Note
Cost(Eco^{LAMP})	1	235	Components:25, Edges: 110
Cost(Eco^{WAMP})	1	228	Components:24, Edges: 104
Cost(Eco^{MEAN})	1	144	Components:8, Edges: 36
Cost(Eco^{MERN})	1	234	Components:16, Edges: 118
Cost(Eco^{MEVN})	1	203	Components:13, Edges: 90

Table 3. Use Case 3: Ecosystem Maintenance Cost of Software Stack.

5. Discussion

Maintaining a software ecosystem can be a costly task if frequent updates must be impact evaluated. Figure 1 illustrates the components and complex potential dependencies of the Python software ecosystem based on the last 50 versions released. Generally, having a component and graph based overview simplifies documenting and determining the update process. The weighted maintenance cost function allows adaption to different Industry 4.0 ecosystem application use cases. The use cases 1-3 illustrate how software release notes metadata can be leveraged to address various real ecosystem questions and use cases. The results return clear data driven results on how to answer the questions. For example, in use case 2, picking the MEAN stack means a smaller and less connected ecosystem, which generally implies less software update efforts on an ecosystem level. The decision can be different, if answered by software component technical maintenance experts only. To complement the experience of software component technical maintenance experts, leveraging release notes data is a potential supplemental approach to arrive at the ecosystem maintenance costs. Towards the goal of reducing the maintenance cost, generally, applying open source software components potentially allows the fast reuse of software functionality, but frequent updates may potentially

add a significant maintenance cost at the ecosystem level. As a result, generally, reviewing the maintenance cost of open source software components or stacks may reduce the overall ecosystem maintenance cost. With regard to limitations, the following points shall be considered. First, the results can only partially leverage release notes data, which does not apply semantic versioning. More specifically, while the update frequency can be considered, use cases that require weighting major, minor, or patch versions are not supported. Second, the maintenance cost results are relative values that are neither validated in a real ecosystem environment, normalized, nor mapped to actual effort metrics. Third, links are based on shared semantic description tags and indicate general association. Ideally, links indicate dependencies, such that ecosystem teams must coordinate software updates. Based on these limitations the next avenue for further studies includes verifying model in a real Industry 4.0 software ecosystem setting. In addition leveraging real software ecosystem offer valuable feedback towards the normalization of maintenance cost. Assuming that dependency links are the most effort related types of links, lastly, more effort with regard to reducing graphs towards dependency links is required.

6. Conclusion

Impact evaluating the maintenance cost of software updates is an important and challenging task for many Industry 4.0 domains that are often embedded in software ecosystems. The hypothesis of this paper is that software release notes metadata can be leveraged to support this task. Towards this objective, section 3 proposed a graph-based model, section 4 presented prototype implementation use cases followed by a discussion in section 5. The use cases clearly show that early maintenance cost results can be derived by comparing the different variants. These results are significant with regard to the metadata applied which is not limited to organizations, projects, or other general constraining factors. While the use case scenarios applied are common across many Industry 4.0 domains, future work includes validating the model, prototype, and use cases in a real Industry 4.0 setting.

Acknowledgements

We would like to thank NIST NVD, GitHub, Linux Distrowatch Feed, and Wikipedia for providing public access to the software update release metadata.

References

[1] M. Jovanovic, M. Antonia, A.L. Mesquida, B. Lalic, Transition of organizational roles in Agile transformation process: A grounded theory approach, J. of Systems and Software, Volume 133, 2017, Pages 174-194, ISSN 0164-1212, doi:10.1016/j.jss.2017.07.008.

[2] P. Diebold, Z. Thomas, D. Richter, How do agile practices support automotive SPICE compliance?, In Proc. of the 2017 Intl. Conf. on Software and System Process (ICSSP 2017), Association for Computing Machinery, New York, NY, USA, 80–84, doi:10.1145/3084100.3084108.

[3] S. Kurkovsky, C. Williams, Raspberry Pi as a Platform for the Internet of Things Projects: Experiences and Lessons, In Proc. of the 2017 ACM Conf. on Innovation and Technology in Computer Science Education (ITiCSE '17), Association for Computing Machinery, New York, NY, USA, 64–69, doi:10.1145/3059009.3059028.

[4] C. Martin, D. Garcia-Swartz, From Mainframes to Smartphones: A History of the Intl. Computer Industry, Cambridge, Massachusetts; London, England: Harvard University Press. http://www.jstor.org/stable/j.ctvjf9wmr, 2015 (accessed 08.01.23).

[5] M. Chakroun, J. Sayah, C. Kallab, S. Haddad, Internet of Things: Towards a Solid Ecosystem of Interconnected Things, Advances in Internet of Things, 12, 35-64, doi:10.4236/ait.2022.123004.

[6] D. Martens, W. Maalej, Release Early, Release Often, and Watch Your Users' Emotions: Lessons From Emotional Patterns, IEEE Software, vol. 36, no. 5, pp. 32-37, Sept.-Oct. 2019, doi:10.1109/MS.2019.2923603.

[7] M. Clark, A new Windows 10 update fixes the issue where some printers would crash computers. https://theverge.com/2021/3/15/22332916/windows-10-update-fixes-printer-blue-screen-of-death-issue-crash-microsoft, 2021 (accessed 08.01.23).

[8] D. Massanori, B.P. Cafeo, I. Wiese, A. Fontão, Death of a Software Ecosystem: a Developer Relations (DevRel) perspective, In Proc. of the XXXIV Brazilian Symposium on Software Engineering (SBES '20), Association for Computing Machinery, New York, NY, USA, 399–404, doi:10.1145/3422392.3422445.

[9] V. Prakash, S. Xie, D.Y. Huang, Inferring Software Update Practices on Smart Home IoT Devices Through User Agent Analysis, In Proc. of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED'22), Association for Computing Machinery, New York, NY, USA, 93–103, doi:10.1145/3560835.3564551.

[10] U.K. Singh, C. Joshi, D. Kanellopoulos, A framework for zero-day vulnerabilities detection and prioritization, J. Inf. Secur. Appl. 46, C (Jun 2019), 164–172, doi:10.1016/j.jisa.2019.03.011.

[11] W. Torbacki, A Hybrid MCDM Model Combining DANP and PROMETHEE II Methods for the Assessment of Cybersecurity, Industry 4.0. Sustainability 2021, 13, 8833, doi:10.3390/su13168833.

[12] A.K. Jaiswal, Apache Log4J Shell Vulnerability Analysis, Intl. Journal of Engineering Applied Sciences and Technology, 2022 Vol. 7, Issue 6, ISSN No. 2455-2143, Pages 396-404. https://www.ijeast.com/papers/396-404,%20Tesma0706,IJEAST.pdf, 2022 (accessed 08.01.23).

[13] A.M. Algarni, The Historical Relationship between the Software Vulnerability Lifecycle and Vulnerability Markets: Security and Economic Risks, Computers. 2022; 11(9):137, doi:10.3390/computers11090137.

[14] M. Korolov, Critical VMware vSphere Vulnerability Is a Must-Patch. https://www.datacenterknowledge.com/security/critical-vmware-vsphere-vulnerability-must-patch, 2021 (accessed 08.01.23).

[15] J. Zhou, R.J. Walker, API deprecation: a retrospective analysis and detection method for code examples on the web, In Proc. of the 2016 24th ACM SIGSOFT Intl. Symposium on Foundations of Software Engineering (FSE 2016). Assoc. for Computing Machinery, New York, NY, USA, 266–277, doi:10.1145/2950290.2950298.

[16] C. Bogart, C. Kästner, J. Herbsleb, F. Thung, How to break an API: cost negotiation and community values in three software ecosystems, In Proc. of the 2016 24th ACM SIGSOFT Intl. Symposium on Foundations of Software Engineering (FSE 2016). Assoc. for Computing Machinery, New York, NY, USA, 109–120, doi: 10.1145/2950290.2950325.

[17] D. Jayasuriya, Towards Automated Updates of Software Dependencies, In Companion Proc. of the 2022 ACM SIGPLAN Intl. Conf. on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion 2022). Assoc. for Computing Machinery, New York, NY, USA, 29–33, doi:10.1145/3563768.3565548.

[18] S. Berhe, M. Maynard, F. Khomh, Software Release Patterns When is it a good time to update a software component?, Procedia Computer Science, Volume 170, 2020, Pages 618-625, ISSN 1877-0509, doi:10.1016/j.procs.2020.03.142.

[19] R. Cox, Surviving Software Dependencies, Comm. ACM 62, 9 (September 2019), 36–43, doi:10.1145/3347446.

[20] R.D. Banker, S.M. Datar, C.F. Kemerer, D. Zweig, Software Complexity and Maintenance Costs, Commun. ACM 36, 11 (Nov. 1993), 81–94, doi:10.1145/163359.163375.

[21] K. Crowston, K. Wei, J. Howison, A. Wiggins, Free/Libre open-source software development: What we know and what we do not know., ACM Comput. Surv. 44, 2, Article 7 (February 2012), 35 pages, doi:10.1145/2089125.20891275.

[22] E.E. Ogheneovo, On the Relationship between Software Complexity and Maintenance Costs, J. of Computer and Communications, 2, 1-16, doi:10.4236/jcc.2014.214001.

[23] T. Doran, IEEE/ISO/IEC 14764-2021, ISO/IEC/IEEE Intl. Standard - Software engineering - Software life cycle processes - Maintenance. https://standards.ieee.org/ieee/14764/7701/, 2021 (accessed 08.01.23).

[24] T. Doran, Systems and Software Engineering - Systems and Software Assurance - Part 3: System Integrity Levels. https://development.standards.ieee.org/myproject-web/public/view.html#pardetail/9391, 2021 (accessed 08.01.23).

[25] ISO/IEC/IEEE 15026-1:2019, Systems and software engineering — Systems and software assurance — Part 1: Concepts and vocabulary. https://www.iso.org/standard/73567.html, 2021 (accessed 08.01.23).

[26] D.G. Messerschmitt, C. Szyperski, Software Ecosystem: Understanding an Indispensable Technology and Industry, MIT Press, Cambridge, MA, USA, doi:10.7551/mitpress/6323.001.0001.

[27] M. Lungu, Reverse Engineering Software Ecosystems (Ph.D.), University of Lugano, doi:10.1109/ICSM.2008.4658096.

[28] I.V.D. Berk, J. Slinger, L. Luinenburg, Software ecosystems: a software ecosystem strategy assessment model. In Proc. of the Fourth European Conf. on Software Architecture: Companion Volume (ECSA '10), Assoc. for Computing Machinery, New York, NY, USA, 127–134, doi:10.1145/1842752.1842781.

[29] E. Handoyo, J. Slinger, S. Brinkkemper, Software Ecosystem Modeling: The Value Chains. In Proc. of the Fifth Intl. Conf. on Management of Emergent Digital EcoSystems (MEDES '13), Assoc. for Computing Machinery, New York, NY, USA, 17–24, doi:10.1145/2536146.2536167.

[30] O. Chang, R. Cox, Open Source Vulnerability format (2023), https://ossf.github.io/osv-schema/, 2022 (accessed 08.01.23).

[31] J. Hejderup, A. V. Deursen, G. Gousios, Software Ecosystem Call Graph for Dependency Management, 2018 IEEE/ACM 40th Intl. Conf. on Software Engineering: New Ideas and Emerging Technologies Results (ICSE-NIER), Gothenburg, Sweden, 2018, pp. 101-104, doi:10.1145/3183399.3183417.

[32] N. Kerzazi, F. Khomh, Factors impacting rapid releases: an industrial case study, In Proc. of the 8th ACM/IEEE Intl. Symposium on Empirical Software Engineering and Measurement (ESEM '14), Assoc. for Computing Machinery, New York, NY, USA, Article 61, 1–8, doi:10.1145/2652524.2652589.

[33] F. Reginaldo, S. Gleison, Challenges in Agile Transformation Journey: A Qualitative Study. In Proc. of the XXXIV Brazilian Symposium on Software Engineering (SBES '20), Assoc. for Computing Machinery, New York, NY, USA, 11–20, doi:10.1145/3422392.3422436.

[34] M.L. Drury-Grogan, K. Conboy, T. Acton, Examining decision characteristics and challenges for agile software development. J. System Software 131, C (September 2017), 248–265, doi:10.1016/j.jss.2017.06.003.

[35] P. Chatzipetrou, R. Ouriques, J. Gonzalez-Huerta, Approaching the Relative Estimation Concept with Planning Poker, In Proc. of the 7th Computer Science Education Research Conf. (CSERC '18). Assoc. for Computing Machinery, New York, NY, USA, 21–25, doi:10.1145/3289406.3289409.

[36] K. Moløkken-Østvold, N.C. Haugen, H.C. Benestad, Using planning poker for combining expert estimates in software projects, J. Systems and Software, Vol. 81 (12), 2008,pp. 2106-2117,ISSN 0164-1212, doi:10.1016/j.jss.2008.03.058.

[37] P. Lam, J. Dietrich, D.J. Pearce, Putting the semantics into semantic versioning. In Proc. of the 2020 ACM SIGPLAN Intl. Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2020), Assoc. for Computing Machinery, New York, NY, USA, 157–179, doi:10.1145/3426428.3426922.